

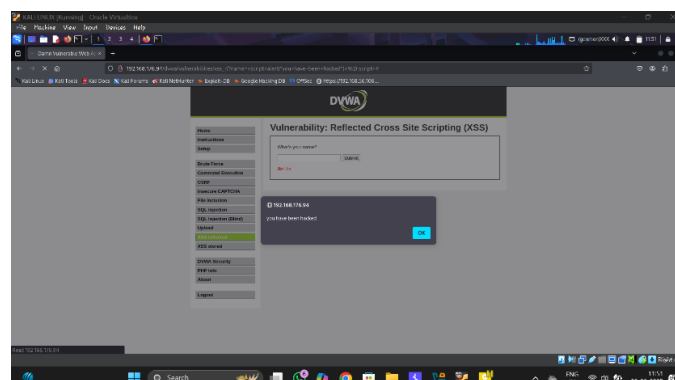
Introduction

This mini research report summarizes experiments on two vulnerabilities in DVWA (Damn Vulnerable Web Application): **Reflected XSS** and **Command Injection**.

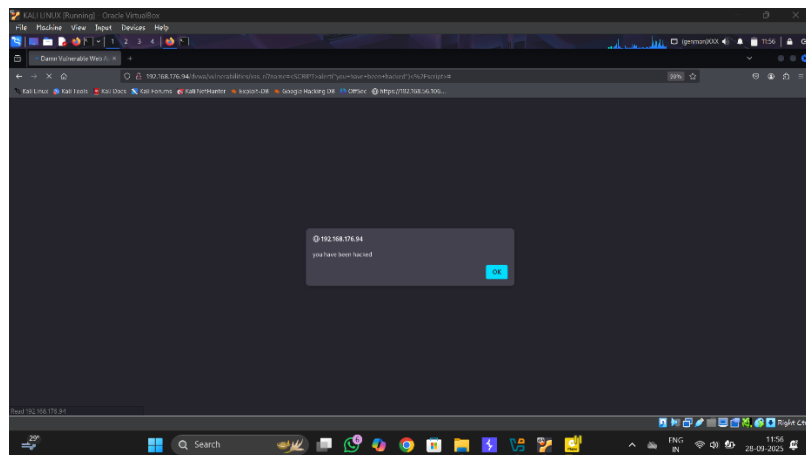
For each vulnerability we list payloads that work at **LOW**, test the same payloads at **MEDIUM/HIGH**, observe differences, and identify the defense mechanism DVWA adds at higher security levels.

Reflected XSS

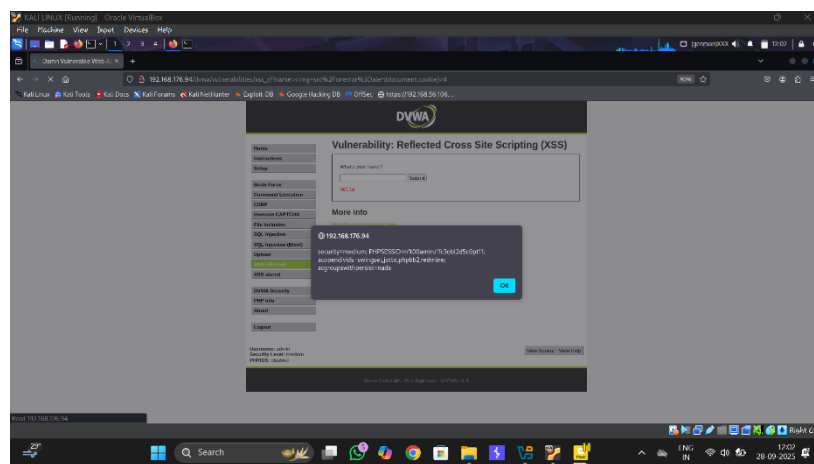
- **Vulnerability:** Reflected Cross-Site Scripting (XSS)
- **Low-level payload (works):**
`<script>alert("you have been hacker")</script>`
- **Medium-level observation:**
The same payload may be filtered or case-normalized.
Example noted: `<SCRIPT>alert("you have been hacker")</script>`
At MEDIUM, DVWA attempts basic filtering or encoding that blocks some script tags, but alternate casing can sometimes bypass it.
- **High-level observation:**
At HIGH, DVWA replaces direct script execution with safer outputs (e.g., stricter HTML-encoding).
Example payload: `<img src onerror=alert(document.cookie)>`
This shows attackers switch to event-handler attributes to test if filters are strict enough.
- **Defense identified:** Input filtering and output encoding (sanitization).
 - **LOW:** DVWA echoes user input raw.
 - **MEDIUM:** Strips or encodes `< >` characters, normalizes case, removes common tags.
 - **HIGH:** Stronger escaping/whitelisting so scripts don't execute.



Security level (low):<script>alert("you have been hacker")</script>



Security level (medium):<SCRIPT>alert("you have been hacker")</script>



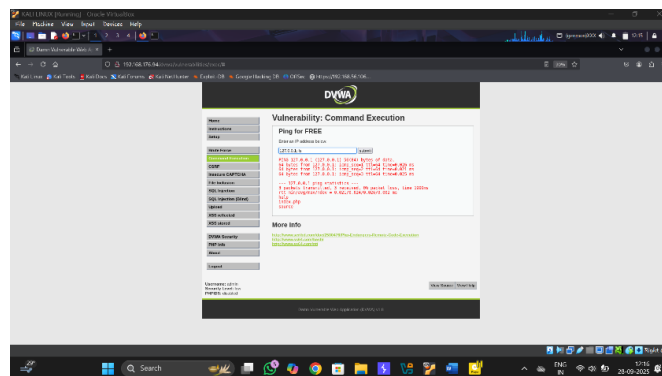
Security level (HIGH):

Command Injection

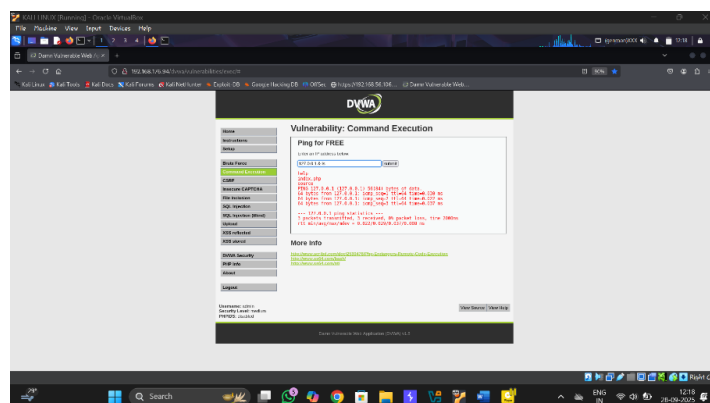
- **Vulnerability:** Command Injection
- **Low-level payload (works):**
127.0.0.1; ls (appends a shell command separated by ;)
- **Medium-level observation:**
Payload example: 127.0.0.1 & ls
At MEDIUM, special characters like ; and & may be blocked or escaped, stopping the injection.
- **High-level observation:**
At HIGH, DVWA validates input strictly (only digits or IP patterns) or uses

parameterized execution rather than direct shell commands.
This prevents metacharacters (; & ||) from being executed.

- **Defense identified:** Input validation/whitelisting and safer command execution.
 - **LOW:** Raw input concatenated into shell.
 - **MEDIUM:** Strips/escapes dangerous operators.
 - **HIGH:** Strict validation, or safe APIs to execute without shell injection.



Security level (low): 127.0.0.1; ls



Security level (medium): 127.0.0.1 & ls

Summary & Recommendations

- DVWA shows how defenses improve step-by-step across **LOW → MEDIUM → HIGH**.
- Common defenses used:
 - Output encoding
 - Input sanitization / whitelisting
 - Escaping special characters
 - Avoiding direct shell commands (using parameterized APIs instead)

